

Table of Contents

1. Cover Page
2. Customer Problem Statement
3. System Requirements
4. Functional Requirement Specification
5. System Sequence Diagram
6. Activity Diagram
7. User Interface Specification
8. Traceability Matrix
9. System Architecture and System Design
10. Algorithms and Data Structures
11. User Interface Design and Implementation
12. Design of Tests
13. Project Plan
14. References

1. Cover Page

Project Title: Clay'sCars.com
Author: Clay Vaughn
Date: May 2025

2. Customer Problem Statement

Local residents in Madison, Indiana have difficulty comparing used car inventory across various dealerships.
Each dealer maintains its own website, leading to fragmented, inconsistent user experiences and scattered data.
There is a need for a centralized platform to present available cars for sale in a clean, user-friendly interface
with automation to extract up-to-date listings from multiple local dealerships.

3. System Requirements

- The system shall provide a responsive website viewable on mobile and desktop devices.
- The system shall present car listings from at least 7 dealerships in Madison, Indiana.
- The system shall automate car data collection via web scraping tools.
- The system shall offer a simplified dropdown search by make and model.
- The system shall remove unnecessary navigation tabs (e.g., Inventory, Dealers).
- The system shall exclude contact forms and focus purely on browsing.
- The system shall operate with minimal user input to complete search tasks.

4. Functional Requirement Specification

FR1: The system shall allow users to filter vehicles by make and model using a dropdown menu.
FR2: The system shall fetch and refresh vehicle listings from partner dealership sites.
FR3: The system shall display vehicle cards with image, year, price, mileage, and dealer name.
FR4: The system shall support responsive layouts and optimize performance for mobile.

FR5: The system shall highlight automation by minimizing required user input.

5. System Sequence Diagram

[Diagram Placeholder: Will be added in the final PDF – shows user -> UI -> backend scraper -> car listings]

6. Activity Diagram

[Diagram Placeholder: Will be added in the final PDF – includes activities like “User selects make,” “System filters cars,” “Display listings”]

7. User Interface Specification

The UI consists of:

- A homepage with a dropdown menu for selecting make/model
- A listing area displaying cards of matching vehicles
- Cards contain car image, make/model, year, mileage, dealer logo
- No login required; navigation is streamlined for browsing only
- Optimized for both desktop and mobile formats

8. Traceability Matrix

Requirement ID	Functional Description	UI Element
FR1	Dropdown filtering	Dropdown on homepage
FR2	Web scraping for listings	Backend module
FR3	Display car listings	Vehicle cards
FR4	Responsive mobile layout	CSS / Flexbox grid
FR5	Automation of filtering logic	Auto-populated list

9. System Architecture and System Design

The system follows a modular architecture:

- Front-End: HTML/CSS/JavaScript with dropdown and card layout
- Back-End: Python/Node.js for scraping and serving data
- Deployment: Hosted on GitHub Pages or Vercel (static site) with backend deployed on Render or similar
- Database: Not required; all data is fetched live or cached

10. Algorithms and Data Structures

- Scraper algorithms parse dealership HTML pages, extracting structured data
- Data is temporarily stored in JSON format and rendered dynamically
- Filtering is implemented using JS logic and dropdown selection
- Rate-limiting and retry mechanisms ensure scraping stability

11. User Interface Design and Implementation

The site emphasizes:

- Minimalist design to avoid distractions
- One-click dropdown for immediate filtering
- Auto-layout vehicle cards with consistent sizing
- Adaptive design for different screen widths
- Font and image optimization for fast loading

12. Design of Tests

Unit Tests:

- Scraper parsing functions
- Dropdown behavior and filtering logic

Integration Tests:

- End-to-end test from dropdown selection to correct listing rendering
- Layout consistency check on desktop vs mobile

Manual Tests:

- Verify updated listings every 24 hours
- Validate image loading and card rendering per dealership

13. Project Plan

Week 1–2: Requirements Gathering, Site Layout Design

Week 3–4: Build Front-End Structure, Add Car Cards

Week 5: Web Scraping Development for 7 Dealerships

Week 6: Mobile Optimization, Testing, and Bug Fixes

Week 7: Prepare Brochure, Documentation, Presentation

Tools Used: GitHub, Python, JavaScript, HTML/CSS, Puppeteer/Cheerio

14. References

- Dealership websites (e.g., chandlerchevcad.com, madisonfordin.com)
- MDN Web Docs for HTML/CSS/JS
- GitHub Pages deployment guide
- Puppeteer web scraping documentation